

ANALISIS PRELIMINAR

WEB

CASA RURAL

Quiero que generes la **arquitectura funcional, estructura de proyecto, pantallas, componentes, rutas y base técnica inicial** de una aplicación web profesional para una **casa rural llamada “La Rasilla”, situada en Valles Pasiegos**, preparada para desplegar en **Vercel**.

Stack obligatorio

- **Frontend:** React + TypeScript
 - **Build:** Vite
 - **Routing:** React Router
 - **Backend:** Supabase
 - **Autenticación:** Supabase Auth
 - **Base de datos:** Supabase Postgres
 - **Lógica backend:** solo **Edge Functions**, no usar RPC
 - **Pagos:** Stripe Checkout + webhooks
 - **Deploy:** Vercel
 - **Código organizado dentro de src/**
 - Diseño profesional, moderno, limpio, tipo alojamiento premium rural
 - Preparado para desarrollo en **VS Code**
-

Objetivo del proyecto

Construir una **web pública con motor de reservas** y una **zona privada de administración** para gestionar la casa rural **La Rasilla**.

La casa se alquila **íntegramente**, no por habitaciones sueltas.

Datos del negocio

- Nombre: **La Rasilla**
- Ubicación: **Valles Pasiegos**
- Tipo: **Casa rural de alquiler íntegro**
- Distribución:
 - **5 habitaciones dobles**
 - **1 de ellas con posibilidad de triple**

- Capacidad:
 - **10 huéspedes incluidos**
 - **máximo 11 huéspedes**
 - No escalar a PMS complejo. Debe ser una solución propia, simple y controlada.
-

Reglas de precio

Implementar en la lógica del sistema estas temporadas y reglas:

Temporada BASE

- 275 €/noche
- +27,5 € por huésped extra / noche
- +50 € limpieza

Temporada ALTA

- 300 €/noche
- +30 € por huésped extra / noche
- +50 € limpieza

Fórmula

$\text{total} = (\text{precio_noche} * \text{noches}) + (\text{extra_huesped} * \text{noches si aplica}) + \text{limpieza}$

Tarifas

1. No reembolsable

- 10% de descuento sobre alojamiento
- la limpieza no se descuenta
- pago total por adelantado

2. Flexible

- precio normal
 - política de cancelación aplicada
 - opción recomendada: pago del 30% como señal
-

Política de cancelación flexible

Aplicar esta lógica:

- cancelación con **60 días o más**: reembolso 100%

- entre **45 y 59 días**: reembolso 50%
- entre **30 y 44 días**: reembolso 25%
- con **menos de 30 días**: reembolso 0%

El usuario puede cancelar según política, pero **no puede modificar su reserva**.

El administrador sí puede modificar, recalcular, cancelar y reenviar confirmaciones.

Sincronización externa

La aplicación debe estar preparada para sincronizar disponibilidad vía **iCal** con:

- Booking
- Airbnb
- Escapada Rural

Reglas iCal

- importar bloqueos/ocupación
 - exportar feed propio
 - usar iCal **solo para disponibilidad**
 - **no** usar iCal para precios, pagos ni reglas
-

Zona pública

Crear una web pública completa, orientada a conversión, con estas secciones y páginas:

1. Home

Debe incluir:

- hero principal con imagen potente de la casa
- propuesta de valor clara
- buscador rápido de disponibilidad
- CTA para reservar
- resumen visual de la casa
- bloque de opiniones de viajeros
- bloque de servicios
- galería destacada
- acceso a contacto

- diseño emocional, elegante y rural premium

2. La Casa

Página detallada con:

- historia o presentación del alojamiento
- descripción completa
- capacidad
- distribución de habitaciones
- normas básicas
- estancia mínima
- máximo de huéspedes
- información de precios orientativos
- ventajas de reservar en web oficial

3. Galería / Fotos

- grid profesional
- categorías: exterior, interior, habitaciones, cocina, salón, entorno
- lightbox o visor de imágenes
- diseño visual muy cuidado

4. Servicios

Mostrar servicios y comodidades:

- casa completa
- cocina equipada
- salón
- baños
- calefacción
- wifi
- aparcamiento si aplica
- entorno natural
- cualquier otro servicio destacable

5. Opiniones

- testimonios visuales
- puntuaciones

- tarjetas de reseñas
- preparado para datos reales o mock inicial
- diseño creíble, no genérico

6. Contacto

- formulario de contacto
- datos de contacto visibles
- teléfono
- email
- ubicación / mapa o bloque de situación
- CTA a WhatsApp opcional
- validación del formulario

7. Disponibilidad / Reservar

Pantalla pública central del motor de reservas con:

- selector de entrada
- selector de salida
- selector de huéspedes
- validación de fechas
- días ocupados en gris
- calendario mensual visual
- no permitir seleccionar fechas ocupadas
- mostrar disponibilidad en tiempo real
- cálculo de noches
- mostrar las dos tarifas disponibles
- mostrar precio desglosado:
 - alojamiento
 - suplemento por huésped extra
 - limpieza
 - descuento si es no reembolsable
 - total final
- CTA para reservar y pagar

8. Política de cancelación / condiciones

Página pública con:

- condiciones de reserva
 - política de cancelación
 - política de pago
 - horarios de entrada/salida
 - normas esenciales
-

Flujo de reserva

Implementar el flujo funcional completo:

1. usuario selecciona fechas y huéspedes
 2. sistema valida disponibilidad
 3. sistema calcula precio
 4. usuario elige tarifa
 5. se crea una **pre-reserva** con estado:
 - PENDING_PAYMENT
 6. se lanza **Stripe Checkout**
 7. Stripe webhook confirma pago
 8. la reserva pasa a CONFIRMED
 9. se bloquea disponibilidad
 10. se envía email de confirmación
-

Estados del sistema

Reserva

- PENDING_PAYMENT
- CONFIRMED
- CANCELLED
- EXPIRED
- opcional en admin: NO_SHOW

Pago

- UNPAID
- PARTIAL
- PAID
- REFUNDED

Origen de reserva

- DIRECT_WEB
 - BOOKING_ICAL
 - AIRBNB_ICAL
 - ESCAPADARURAL_ICAL
 - ADMIN
-

Zona privada admin

Crear una zona admin protegida con login mediante Supabase Auth.

Estructura admin

- /admin/login
- /admin
- /admin/dashboard
- /admin/reservas
- /admin/reservas/:id
- /admin/calendario
- /admin/clientes
- /admin/facturas
- /admin/ingresos
- /admin/configuracion
- /admin/ical
- /admin/emails

Funcionalidades admin

Dashboard

Mostrar:

- reservas del mes

- próximas llegadas
- próximas salidas
- ocupación próxima
- ingresos estimados
- reservas pendientes de pago
- cancelaciones
- resumen por origen de reserva

Calendario admin

Vista tipo calendario con:

- ocupación
- bloqueos
- reservas confirmadas
- reservas pendientes
- origen de la reserva
- posibilidad de abrir ficha de reserva
- modificar fechas desde admin
- recalcular precio
- bloquear manualmente fechas

Lista de reservas

Tabla con filtros:

- por fecha
- por estado
- por origen
- por nombre cliente
- por no presentado
- por pagada / pendiente
- por rango de llegada/salida

Acciones:

- ver reserva
- editar reserva
- cancelar
- marcar no presentado

- reenviar email
- generar factura

Ficha de reserva

Debe permitir:

- ver datos del cliente
- ver tarifa elegida
- ver fechas
- ver número de huéspedes
- ver importes
- ver estado reserva y pago
- modificar fechas
- recalcular precio
- cancelar reserva
- registrar reembolso
- reenviar confirmación
- enviar modificación por email
- generar / reenviar factura

Gestión de ingresos

- listado de pagos
- importes cobrados
- señal vs pago total
- reservas pendientes de cobro
- filtrado por mes, año, estado y origen
- resumen de ingresos
- diseño claro para control interno

Facturación

- generar factura para clientes
- guardar datos fiscales del cliente
- emitir factura desde reserva
- enviar factura por email
- listado de facturas

- filtro por fecha / cliente / estado

Configuración

- temporadas
- precios base
- precio huésped extra
- limpieza
- estancia mínima
- máximo huéspedes
- textos legales
- datos de contacto
- configuración Stripe
- configuración iCal
- plantillas de email

iCal admin

- alta de feeds externos
- importar/exportar
- ver última sincronización
- ver errores de sincronización
- botón de sincronización manual
- feed propio de exportación

Estructura técnica que quiero que generes

Quiero que propongas una **estructura realista de carpetas** para React + TS + Vite dentro de `src/`, separando claramente:

- `public-site`
- `admin`
- `shared`
- `components`
- `layouts`
- `routes`
- `hooks`

- lib
- services
- types
- styles

Ejemplo esperado

```
src/  
  app/  
    routes/  
  public/  
    pages/  
    components/  
    layouts/  
  admin/  
    pages/  
    components/  
    layouts/  
  shared/  
    components/  
    ui/  
    utils/  
    constants/  
    hooks/  
    types/  
  services/  
  integrations/  
    stripe/  
    supabase/  
    ical/  
  context/  
  assets/
```

Requisitos de implementación

Quiero que IA Studio genere:

1. Frontend

- React Router configurado
- layouts diferenciados:
 - layout público
 - layout admin
- componentes reutilizables
- diseño responsive
- tipografía elegante
- cards limpias
- calendario visual atractivo

- tablas admin profesionales
- formularios bien resueltos
- feedback visual claro
- loaders, empty states y error states

2. Backend con Supabase

Diseñar tablas necesarias para:

- propiedades / configuración de la casa
- temporadas
- tarifas
- reservas
- huéspedes o titular reserva
- pagos
- facturas
- bloqueos manuales
- feeds iCal
- logs de sincronización iCal
- plantillas email
- auditoría básica

3. Edge Functions

No usar RPC.

Quiero toda la lógica importante en **Supabase Edge Functions**.

Crear la propuesta de funciones para:

- calcular precio de una reserva
- comprobar disponibilidad
- crear pre-reserva
- crear sesión Stripe Checkout
- procesar webhook Stripe
- confirmar reserva
- expirar reservas pendientes
- cancelar reserva
- modificar reserva por admin
- sincronizar iCal import

- generar feed iCal export
- generar factura
- enviar emails transaccionales

4. Stripe

Implementar la arquitectura para:

- pago total de tarifa no reembolsable
- pago parcial flexible (30% señal)
- gestión de estados de pago
- webhook seguro
- relación entre checkout y reserva
- reembolsos desde admin

5. Emails

Preparar sistema para enviar:

- confirmación de reserva
- cancelación
- modificación
- recordatorio previo a llegada
- envío de factura

UX/UI esperado

El diseño debe transmitir:

- confianza
- naturaleza
- alojamiento rural premium
- simplicidad
- facilidad de reserva

Estilo visual

- elegante
- moderno
- limpio

- profesional
- imágenes grandes
- colores cálidos o naturales
- sensación de descanso y escapada rural
- nada recargado
- no parecer plantilla barata

Importante

El motor de reservas debe sentirse claro y muy usable:

- calendario limpio
 - precios entendibles
 - resumen de reserva muy claro
 - proceso corto
 - CTA visibles
-

Restricciones funcionales importantes

- La reserva es de **la casa completa**, no por habitación individual
- Capacidad base: **10 huéspedes**
- Máximo permitido: **11**
- Días ocupados no seleccionables
- El cliente puede:
 - reservar
 - pagar
 - cancelar si su tarifa/política lo permite
- El cliente no puede:
 - modificar la reserva
- El admin sí puede:
 - modificar
 - recalcular
 - cancelar
 - bloquear fechas
 - enviar emails

- emitir facturas
-

Lo que quiero que me entregue IA Studio

Quiero una respuesta estructurada en este orden:

1. **Resumen funcional del producto**
 2. **Arquitectura técnica**
 3. **Estructura de carpetas**
 4. **Mapa de rutas públicas y privadas**
 5. **Listado de páginas**
 6. **Listado de componentes**
 7. **Modelo de datos propuesto**
 8. **Listado de Edge Functions**
 9. **Flujos de usuario**
 10. **Flujos admin**
 11. **Integración Stripe**
 12. **Integración iCal**
 13. **Sistema de emails**
 14. **Roadmap por fases**
 15. **Código base inicial de arranque**
 16. **Recomendaciones de seguridad**
 17. **Recomendaciones de despliegue en Vercel**
 18. **Siguientes archivos a generar**
-

Requisito extra importante

No quiero solo ideas.

Quiero una propuesta **lista para construir**, con enfoque práctico, estructura sólida y decisiones razonables para un proyecto real.

Evita sobreingeniería.

No conviertas esto en un PMS hotelero completo.

Debe ser una solución robusta pero simple, enfocada en:

- web pública
- reservas directas

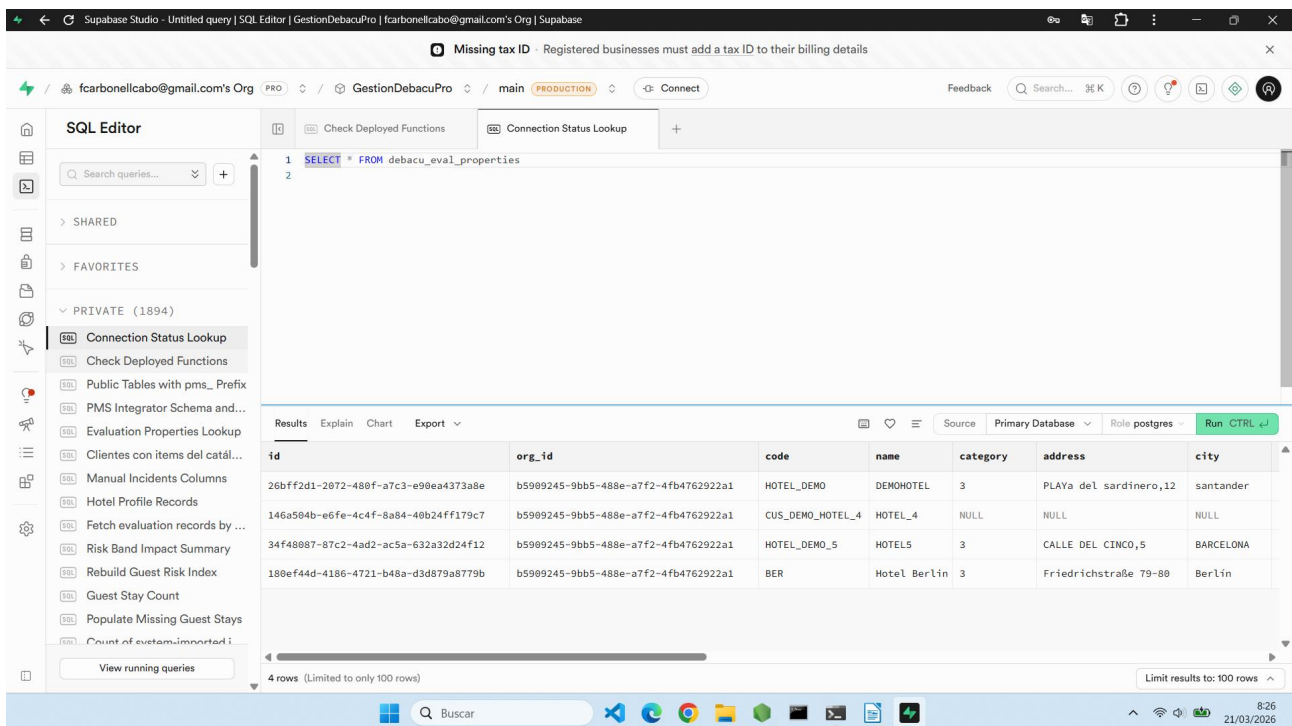
- Stripe
- iCal
- panel admin útil

Instrucción final para IA Studio

Genera la solución con enfoque de producto real, código mantenible y buena UX.
Asume que después voy a pedirte:

- esquema SQL para Supabase
- Edge Functions una a una
- pantallas React reales
- dashboard admin real
- flujo Stripe completo
- integración iCal real

Por tanto, deja la arquitectura preparada para esas siguientes iteraciones.



The screenshot shows the Supabase Studio interface. The SQL Editor is open with the query: `SELECT * FROM debacu_eval_properties`. The results are displayed in a table with 4 rows (limited to 100 rows). The table has columns: id, org_id, code, name, category, address, and city.

id	org_id	code	name	category	address	city
26bff2d1-2972-488f-a7c3-e90ea4373a8e	b5989245-9bb5-488e-a7f2-4fb4762922a1	HOTEL_DEMO	DEMOHOTEL	3	PLAYA del sardinerio,12	santander
146a594b-e6fe-4c4f-8a84-40b24ff179c7	b5989245-9bb5-488e-a7f2-4fb4762922a1	CUS_DEMO_HOTEL_4	HOTEL_4	NULL	NULL	NULL
34f4887-87c2-4ad2-ac5a-632a32d24f12	b5989245-9bb5-488e-a7f2-4fb4762922a1	HOTEL_DEMO_5	HOTEL5	3	CALLE DEL CINCO,5	BARCELONA
188ef44d-4186-4721-b48a-d3d879a8779b	b5989245-9bb5-488e-a7f2-4fb4762922a1	BER	Hotel Berlin	3	Friedrichstraße 79-80	Berlin

En nuestras tablas debemos tener igual esos codigos, entioendo que cuando se creen la primera vez.
Elimino toda referencia a ellos ahora de nuestro sistema